

心得总结：校圈系统

1. 背景与目标

本次开发的目标之一是把项目里的信息订阅能力拓展到 Canvas（同济 Canvas + IAM 登录链路），实现“绑定账号 → 同步课程/公告 → 写入信息中心”的闭环。理想情况是后端通过 HTTP 模拟登录后抓取页面，但实际接入过程中遇到学校统一认证（IAM）的风控策略，导致链路并非完全可控。

2. AI 辅助编程：我如何用它提效

这次我对 AI 辅助编程的体会不是“让它替我写代码”，而是让它在三个环节显著提速：

2.1 快速定位问题：把“感觉不对”变成“证据链”

很多问题初看像玄学，例如：

- 前端请求返回 200 但拿到的是 `<!DOCTYPE html>`（实际打到了前端 dev server）
- Canvas 同步失败但不清楚卡在哪一步
- IAM 提示“无登录认证用户信息”

AI 帮我做的关键是：把问题拆成可观测的链路，并用日志把每一步固定下来（状态码、location、cookie、表单字段等）。有了结构化诊断后，“猜”变成了“对照”。

2.2 快速迭代：小改动 + 立刻验证

我学习到：不要一次改很多逻辑，而是每次只改一个点（例如 cookie 保存、表单字段保序、入口路径），然后通过日志立即验证是否推进到下一步。AI 在这里的价值是能快速给出多种可能性、并把改动落到可执行的代码 patch。

2.3 补齐工程能力：把功能做成“能用、能排障、能维护”

除了功能本身，还需要：

- 失败时给出明确错误类型（例如 `CAPTCHA_REQUIRED`）
- 前端能根据错误类型引导用户完成下一步（例如触发浏览器登录再重试）
- 后端日志清晰且可检索（`CANVAS_DIAG` 按行输出）

AI 在工程收尾（可观测性、错误分层、前后端协作协议）上帮助很大。

3. 风险管理：第三方系统不可控是最大的风险源

这次最大的风险不是“代码写错”，而是**第三方系统的认证策略不可控**。

3.1 风险点：IAM 验证码 / 二次认证导致 HTTP 模拟不可靠

在日志里我们明确看到了 IAM 表单包含 `j_checkcode`（验证码字段）。这种情况下：

- 纯 `HttpClient` 提交用户名/密码很容易被判定为“上下文不完整”

- 最终会出现 IAM 错误页：无登录认证用户信息

结论：只靠 HTTP 模拟在风控开启时无法稳定跑通，这是**产品级风险**，不是“再调几个字段”就能解决的。

3.2 风险应对：设计降级路径，让系统“可恢复”

我们采取的策略不是死磕 HTTP，而是把风险显式化并提供可行绕行：

- 后端检测到验证码 → 直接返回更准确的状态 `CAPTCHA_REQUIRED`
- 引入 Playwright（真浏览器）进行一次“人工介入登录”
- 登录成功后导出 cookie 存库，后续同步优先复用会话，避免频繁触发验证码
- 前端在同步失败时弹窗引导用户“去浏览器登录”，完成后自动重试同步

这样做的意义是：即使第三方系统不可控，我们仍能把流程变成“可解释、可操作、可恢复”。

3.3 风险边界：能力范围与合规意识

验证码本质是对自动化行为的限制。项目层面我们选择“用户自己完成验证码/MFA”，而不是 OCR/打码平台等自动破解手段，避免引入更高的合规风险与不稳定性。

4. 功能点进度管理：按闭环推进，而不是按模块堆叠

我这次更明显感受到“进度管理”应以用户闭环为单位：

4.1 先打通最短闭环

- 绑定接口可用（保存 baseUrl/账号/密码）
- 同步接口能跑（哪怕失败也要给诊断）
- 前端能触发同步并看到明确结果

4.2 再补齐关键基础设施

- 诊断日志：把每一步跳转、cookie、表单字段变成可查信息
- 会话保存：浏览器登录后 cookie 入库；同步时优先复用会话
- 错误分型：区分“需要验证码”与“代码/网络错误”，避免前端只看到一长串难懂的报错

4.3 再完善体验与一致性

- 前端展示乱码修复：列表卡片与详情页保持一致
- 同步按钮体验：遇到 `CAPTCHA_REQUIRED` 时给出明确引导，而不是让用户面对一大段技术日志

5. 反思与改进方向

5.1 我学到的

- AI 能显著加速“定位—验证—迭代”的循环，但前提是我提供足够上下文（日志、现象、目标）
- 风险不是上线前才处理，而是在方案设计时就要准备降级路径
- 功能交付不等于“代码写完”，还需要可观测性和可恢复性

5.2 下一步可以做得更好

- 把浏览器登录做成更可控的流程（例如明确提示超时、明确显示当前状态）
 - 把投票/Canvas 等功能的状态展示做成更产品化的反馈（成功/失败原因更短更清晰）
 - 对第三方依赖建立“能力矩阵”：哪些能自动化、哪些必须人工介入、哪些不可支持
-

6. 总结

这次项目让我更明确：AI 辅助编程的价值不只是写代码，而是帮助我快速形成“证据链”和“可验证改动”；而风险管理的核心是承认第三方系统不可控，并把它转化成清晰的产品流程（检测 → 提示 → 降级/绕行 → 恢复）。最终的交付标准不只是“能跑”，还要“出问题时能解释、能恢复”。